- **Activity 1.3: Validate Gemini API connectivity**
  - Make a sample prompt call using Python to confirm integration.

```python
from google import genai
from google.genai import types
from PIL import Image
from io import BytesIO
import base64

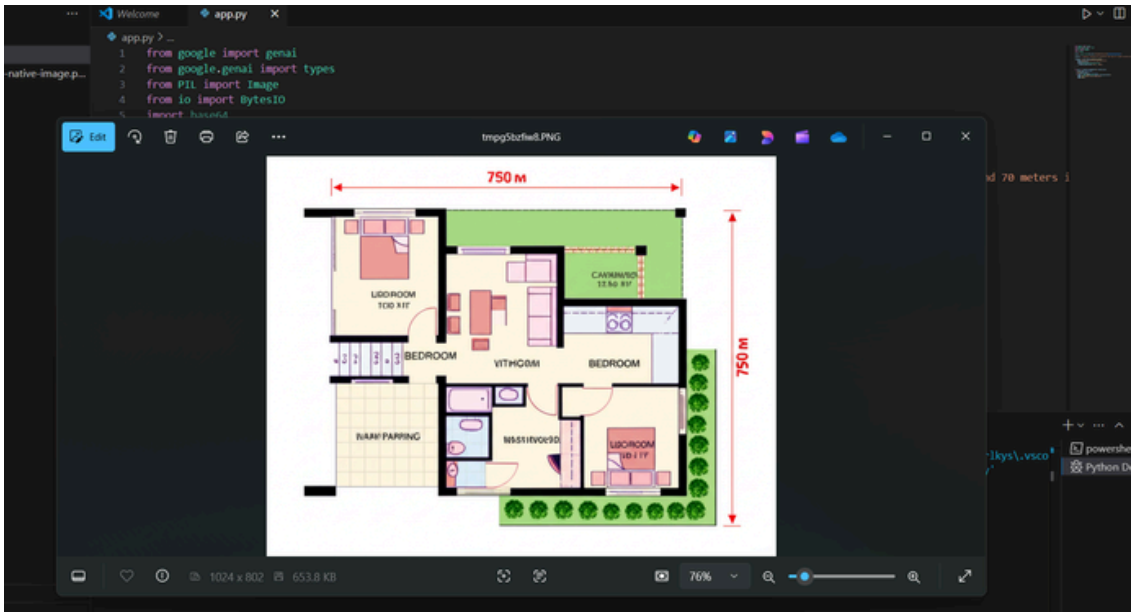
client = genai.Client(api_key="AIzaSyDdzTxAIisLXlQ")

contents = ('Generate a detailed 2D architectural floor plan for a modern two-story house with a total area of 150 meters in length and 70 meters

response = client.models.generate_content(
    model="gemini-2.0-flash-exp-image-generation",
    contents=contents,
    config=types.GenerateContentConfig(
      response_modalities=['Text', 'Image']
    )
)

for part in response.candidates[0].content.parts:
  if part.text is not None:
    print(part.text)
  elif part.inline_data is not None:
    image = Image.open(BytesIO((part.inline_data.data)))
    image.save('gemini-native-image.png')
    image.show()
```

  - Test both text-only and image + text prompts to ensure multimodal compatibility.

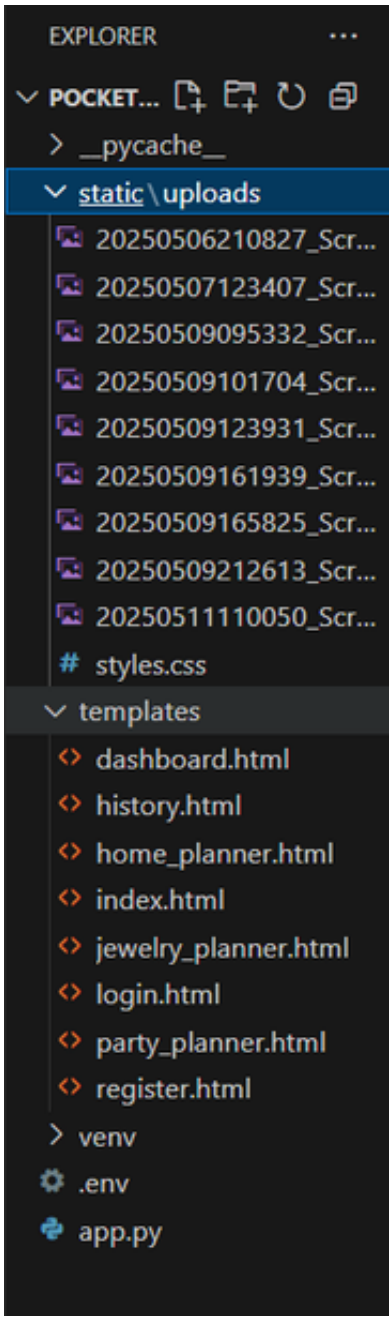


  -

MILESTONE 2: Core Functionalities Development

This milestone focuses on building the **core backend intelligence** of **PocketSmart AI** using the Flask framework and Google's **Gemini 1.5 Flash Pro** foundation model. The primary goal is to develop a robust and modular system capable of accepting user budgets (and optional image inputs), intelligently interpreting the context, and returning domain-specific, platform-aware recommendations.The backend handles prompt generation, API communication with Gemini, modular planning logic (for Home, Party, and Jewelry), and optional fallback strategies when product matches are limited. All core logic is abstracted into a utility service file (gemini_utils.py) that handles prompt orchestration, budget formatting, domain segmentation, and image analysis for multimodal tasks.

File Explorer Structure



Activity 2.1: Fast API Instilization and import libraries

- Import all required libraries for the project

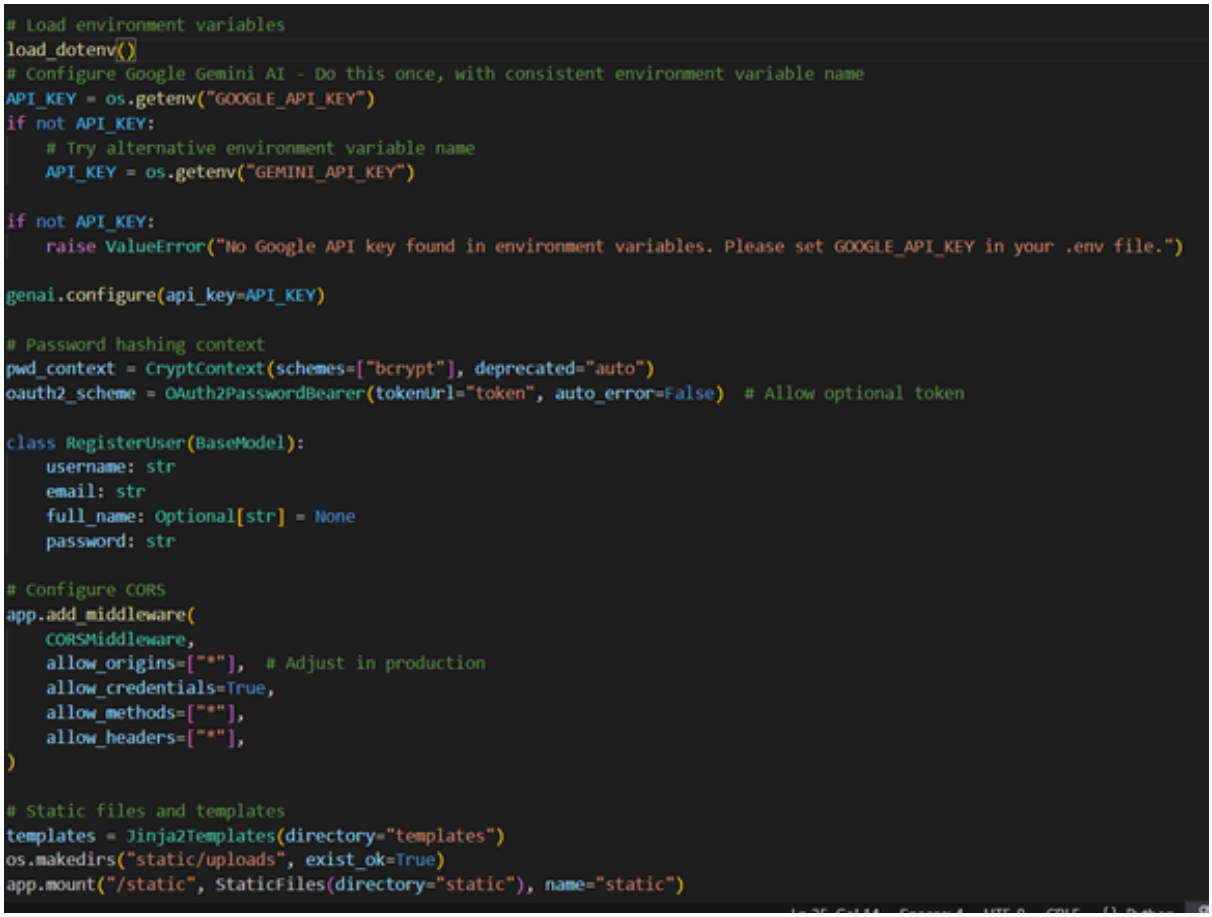
```python
import os
import json
import re
import base64
import shutil
import urllib.parse
from fastapi import FastAPI, HTTPException, Depends, File, UploadFile, Form, Request, status, Cookie
from fastapi.responses import JSONResponse, RedirectResponse
from fastapi.middleware.cors import CORSMiddleware
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from fastapi.responses import HTMLResponse
from typing import List, Optional, Dict, Any, Union
from pydantic import BaseModel, EmailStr
from passlib.context import CryptContext
from jose import JWTError, jwt
from datetime import datetime, timedelta
from PIL import Image
from io import BytesIO
import google.generativeai as genai
from dotenv import load_dotenv
import asyncio
from typing import Dict, Any, List, Optional
import uuid
```

- Fast API Initialization

```python
# FastAPI app initialization
app = FastAPI(title="PocketSmart: AI Budget Planner")

SECRET_KEY = os.getenv("SECRET_KEY", "your_secret_key")
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30
```

- Load .env environment



```python
# Load environment variables
load_dotenv()
# Configure Google Gemini AI - Do this once, with consistent environment variable name
API_KEY = os.getenv("GOOGLE_API_KEY")
if not API_KEY:
    # Try alternative environment variable name
    API_KEY = os.getenv("GEMINI_API_KEY")

if not API_KEY:
    raise ValueError("No Google API key found in environment variables. Please set GOOGLE_API_KEY in your .env file.")

genai.configure(api_key=API_KEY)

# Password hashing context
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token", auto_error=False)  # Allow optional token

class RegisterUser(BaseModel):
    username: str
    email: str
    full_name: Optional[str] = None
    password: str

# Configure CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],  # Adjust in production
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Static files and templates
templates = Jinja2Templates(directory="templates")
os.makedirs("static/uploads", exist_ok=True)
app.mount("/static", StaticFiles(directory="static"), name="static")
```

Activity 2.2: Main Functionality code routings

- `/generate-home`: Generates home interior recommendations (furniture, decor, lighting)

```python
@app.get("/home-planner", response_class=HTMLResponse)
async def home_planner(request: Request, current_user: UserInDB = Depends(get_current_active_user)):
    """Home budget planner page"""
    return templates.TemplateResponse("home_planner.html", {"request": request, "user": current_user})
```

- get home_recommendations by using the function

```python
def get_home_recommendations(budget_input: HomeBudgetInput) -> dict:
    """Generate home interior recommendations within budget in INR for Indian market"""
    try:
        # Create prompt for AI model
        prompt = f"""
        I need interior design product recommendations for a home in India with a total budget of ₹{budget_input.total_budget:.2f}.
        Requirements:
        - {budget_input.num_lights} lights/lighting fixtures
        - {budget_input.num_fans} ceiling fans
        - {budget_input.num_furniture} furniture pieces
        - {budget_input.num_dining_tables} dining tables

        Additional rooms to consider:
        {("- Living room" if budget_input.has_living_room else "")}
        {("- Kitchen" if budget_input.has_kitchen else "")}
        {("- Bedroom" if budget_input.has_bedroom else "")}

        Additional requirements: {budget_input.additional_requirements or "None"}

        Please provide a detailed budget breakdown with product recommendations **available in India**.
        Use **Indian brands and pricing**. Include **search terms** suitable for Indian shopping platforms.

        Format your response as JSON with the following structure:
        {{
            "total_budget": {budget_input.total_budget:.2f},
            "budget_breakdown": [
                {{
                    "category": "lighting",
                    "allocation": 0.0,
                    "items": [
                        {{
                            "name": "",
                            "description": "",
                            "estimated_price": 0.0,
                            "quantity": 0,
                            "search_terms": ""
```

- linking the websites like amazon,flipkar,ikea ect…. to get the products

```python
def get_home_recommendations(budget_input: HomeBudgetInput) -> dict:
                        }}
                    ]
                }}
            ],
            "calculation_table": [
                {{
                    "category": "",
                    "items_count": 0,
                    "total_cost": 0.0,
                    "percentage_of_budget": 0.0
                }}
            ],
            "remaining_budget": 0.0,
            "additional_suggestions": []
        }}

        Ensure total costs stay within budget. Include search terms for each item to find on shopping websites like Flipkart, Amazon India, IKEA I
        """

        response = model.generate_content(prompt)
        result = extract_json_from_response(response.text)

        # Add shopping links for each item
        for category in result.get("budget_breakdown", []):
            for item in category.get("items", []):
                search_terms = item.get("search_terms", "")
                if search_terms:
                    item["shopping_links"] = {
                        "amazon": f"https://www.amazon.in/s?k={urllib.parse.quote_plus(search_terms)}",
                        "flipkart": f"https://www.flipkart.com/search?q={urllib.parse.quote_plus(search_terms)}",
                        "ikea": f"https://www.ikea.com/in/en/search/?q={urllib.parse.quote_plus(search_terms)}",
                        "myntra": f"https://www.myntra.com/search?q={urllib.parse.quote_plus(search_terms)}",
                        "ajio": f"https://www.ajio.com/search/?text={urllib.parse.quote_plus(search_terms)}"
                    }

        return result
```

```
    except Exception as e:
        raise HTTPException(500, f"Error generating recommendations: {str(e)}")


def usd_to_inr(amount_usd: float, exchange_rate: float = 83.0) -> float:
    """Convert USD amount to INR using the specified exchange rate"""
    return amount_usd * exchange_rate
```

- `/generate-party`: Suggests catering, venue, and decoration options for events

```
@app.get("/party-planner", response_class=HTMLResponse)
async def party_planner(request: Request, current_user: UserInDB = Depends(get_current_active_user)):
    """Party budget planner page"""
    return templates.TemplateResponse("party_planner.html", {"request": request, "user": current_user})
```

- get home_recommendations by using the function